



32

Drift report + alerting (Evidently)

US3.5 · C3/C8

■ Objectifs & déroulé de la séance

À l'issue, vous serez capable de :

- › Générer un rapport de drift automatisé
- › Intégrer l'étape drift au flow (après predict)
- › Définir une règle d'alerte + enregistrer l'événement
- › Limiter le bruit (cooldown / hystérésis)

Déroulé de la séance

- 1 **Théorie les concepts clés**
- 2 **Travaux dirigés** TP guidé, correction au fil de l'eau
- 3 **Autonomie tutorée** vous avancez, formateur dispo
- 4 **Bilan synthèse + preuves**

PREUVE FINALE rapport généré · alerte simulée + drift_events · test 0
→ 1 → 0

1

Module 32 — les concepts clés

Théorie

■ Scénario atelier & enjeux

Tu ne vas pas fixer un graphe de drift toute la journée. Le système doit t'alerter tout seul quand ça dérive — sans te spammer pour un faux positif.

Angle éthique / risque (C2) Trop d'alertes = fatigue d'alarme = on finit par rater la vraie. Le cooldown est un enjeu de fiabilité humaine.

- Éco-conception : calculer le drift à la bonne fréquence (et non en continu inutilement) économise du calcul.

■ Contexte & outils — pourquoi ce module

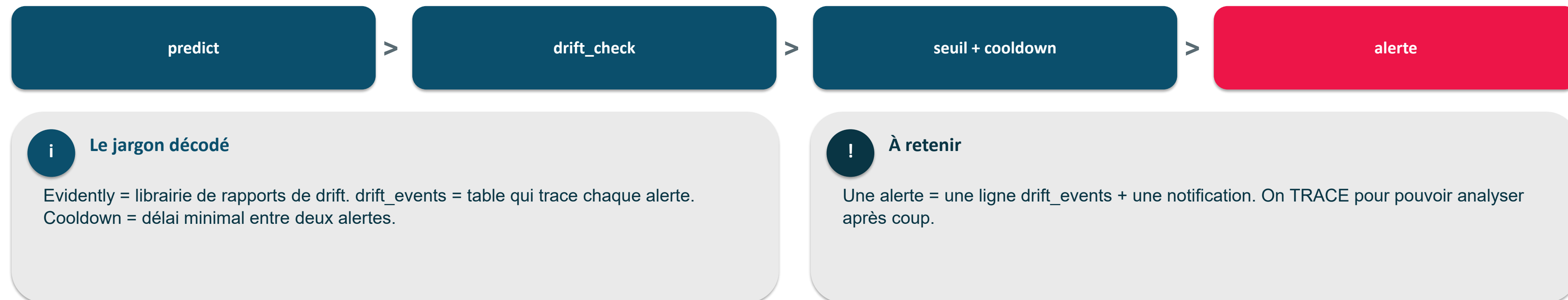
Contexte : détecter ne suffit pas. Pourquoi : on PRODUIT un rapport de dérive et on alerte sans fatiguer l'équipe.

- Evidently : génère des rapports de data drift
- Cooldown / hystérésis : évite les alertes qui clignotent (anti-flapping)
- Table drift_events : on garde la trace des alertes
- KPI technique vs métier : ce qui compte vraiment

Objectif du module Une alerte de dérive fiable, traçable et non spammante.

■ Du calcul au rapport automatisé... intégré au flow

On emballe le calcul de drift en rapport (HTML + JSON) généré à chaque run, après la prédiction.



EN UNE PHRASE L'étape drift se branche APRÈS predict dans le flow Prefect.

■ La fatigue d'alerte : le vrai danger

Alerter trop revient à ne pas alerter : on finit par tout ignorer.

- Un drift de 2 jours, flow horaire = 48 alertes identiques → on rate la vraie suivante
- Remèdes : cooldown, hystérésis (réarmer sous 0,1), agrégation par fenêtre

! Attention — piège

Une alerte qu'on ignore ne sert à rien. La qualité d'un système d'alerte se mesure à son SILENCE quand tout va bien.

? Le saviez-vous ?

La « fatigue d'alerte » est un vrai sujet en médecine : à force de bips, les soignants ne réagissent plus. Même cause, même remède.

EN UNE PHRASE Preuve attendue : la séquence $0 \rightarrow 1 \rightarrow 0$ (alerte une fois, puis cooldown).

■ Le drift_check, en vrai (après predict + cooldown)

On branche le contrôle de drift APRÈS predict dans le flow, et on alerte sans spammer.

```
@task
def drift_check(cur, ref, features, cooldown_h=6):
    rep = drift_report(ref, cur, features)          # HTML + JSON
    drifted = [f for f, v in rep.items() if v["drift"]]
    if drifted and not in_cooldown(cooldown_h):
        write_event(drifted); notify()              # ligne drift_events
    return rep
# preuve : sain -> 0 ; +8 °C -> 1 ; relance -> 0 (cooldown)
```

! Attention — piège

Alerter à CHAQUE run = fatigue d'alerte : 48 alertes en 2 j, on rate la vraie suivante.

? Le saviez-vous ?

Même fatigue d'alarme qu'en médecine : la qualité d'un système d'alerte se mesure à son silence.

EN UNE PHRASE Rapport intégré + cooldown/hystérésis = alerter juste. C3/C8.

■ À retenir en 3 points

- Evidently produit le rapport de drift.
- Seuil + cooldown = une alerte utile, pas du spam.
- Données saines → 0 alerte ; dérive → 1 alerte (testé).

2

Module 32 — TP guidé, correction au fil de l'eau

Travaux dirigés

■ TP — drift_check + cooldown

```
@task
def drift_check(cur, ref, features):
    rep = drift_report(ref, cur, features)
    return rep, [f for f,v in rep.items() if v["drift"]]
```

Cooldown Pas de ré-alerte avant N heures sur la même feature.

■ TP — test de l'alerte

```
run_drift(saine)          # 0 alerte  
run_drift(drift_plus8)    # 1 alerte + ligne drift_events  
run_drift(drift_plus8)    # 0 (cooldown)
```

✓ **Preuve attendue** Séquence 0 → 1 → 0 verte ; rapport généré.

3

Module 32 — vous avancez, le formateur reste disponible

Autonomie tutorée

■ Autonomie tutorée — fiabiliser l'alerte

Avancez à votre rythme. Le formateur reste disponible.

- Générer un vrai rapport Evidently HTML et l'ouvrir
- OU ajouter l'hystérésis (réarmer quand $PSI < 0,1$)

+ Posture en distanciel

Repli rapport JSON maison si Evidently traîne en Python 3.13. Montrez votre séquence $0 \rightarrow 1 \rightarrow 0$ pour valider l'anti-bruit.

■ Definition of Done & transition

- DoD : étape drift + rapport + drift_events + test 0 → 1 → 0

Transition → Module 33 On surveille la donnée ; reste à observer le service (Prometheus).